HW2 for CS 6210

You may (and probably should) talk about problems with the each other, with the TA, and with me, providing attribution for any good ideas you might get. Your final write-up should be your own.

```
using LinearAlgebra
```

1. Rewrite for range

Consider the following function for $0 < x < 1$. Write a function `p1mine` that revises `p1ref` to maintain accuracy for $x \ll 1$. For part 2, you may want to look up the `log1p` function in Julia. You may check your answers using arbiitrary precision arithmetic (see `BigFloat` in Julia), but your solutions should use only ordinary floating point.

$$y1 = \frac{(\sqrt{1+x} - \sqrt{1-x})(\sqrt{1+x} + \sqrt{1-x})}{\sqrt{1+x} + \sqrt{1-x}} = \frac{(1+x) - (1-x)}{\sqrt{1+x} + \sqrt{1-x}} = \frac{2x}{\sqrt{1+x} + \sqrt{1-x}}$$

$$y2 = \frac{1}{2}\log\left(1 + \frac{1}{x}\right) - \frac{1}{2}\log\left(\frac{1}{x}\right) = \frac{1}{2}\log\left(\frac{1+1/x}{1/x}\right) = \frac{1}{2}\log(1+x)$$

For small x, derived from Tayler series, $\cos(x) \approx 1 - x^2/2$, $y3 = cos(x) - 1 = -x^2/2$

```
p1ref (generic function with 1 method)
```

```
# Reference formulas (inaccurate for x << 1)
function p1ref(x)
    y1 = sqrt(1+x)-sqrt(1-x)
    y2 = log(sqrt(1+1/x))-log(sqrt(1/x))
    y3 = cos(x)-1
    y1, y2, y3
end
```

```
p1mine (generic function with 1 method)
```

```
function p1mine(x)
    y1 = 2x/ (sqrt(1+x) + sqrt(1-x))
    y2 = 0.5 * log1p(x)
    y3 = - x^2 / 2
    y1, y2, y3
end
```

```
let
    x = 1e-8
    big_x = BigFloat(x)
    big_y1 = sqrt(1+big_x) - sqrt(1-big_x)
    big_y2 = log(sqrt(1+1/big_x))-log(sqrt(1/big_x))
    big_y3 = cos(big_x)-1
    y1, y2, y3 = p1mine(x)
    println(isapprox(y1, big_y1), isapprox(y2, big_y2), isapprox(y3, big_y3))
end
```

```
truetruetrue
```

2. Working backward

Consider the 2-by-2 rotation matrix

$$Q = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}$$

where $c^2 + s^2 = 1$. Note that $Q^TQ = I$. From class, we know that the code `w = Q*v` computes $\hat{w} = (Q + E)v$ where $|E| \lesssim 2\epsilon_{\text{mach}}|Q|$.

- Argue that $\hat{w} = Q\hat{v}$ where $\hat{v} - v = Q^TEv$
- Argue that therefore $\hat{v}$ has a normwise relative error (in the two norm) bounded by $4\epsilon_{\text{mach}}$

From the problem, we know that Q is orthongonal since $Q^TQ = I$, $\text{cond}(Q)_2 = 1$

1. Compute $\hat{w} = Qv + Ev = Q(v + Q^TEv)$. Since $Q^TQ = I$, $Q^T\hat{w} = v + Q^TEv$, $QQ^T = Q^TQ = I$. Thus, $\hat{w} = Q\hat{v}$ with $\hat{v} = v + Q^TEv$, where $\hat{v} - v = Q^TEv$.
2. Since Q is orthongonal, $\hat{v} - v = Q^TEv \implies \|\hat{v} - v\|_2 = \|Q^TEv\|_2 = \|Ev\|_2$, then $\|Ev\|_2 \leq \|E\|_2\|v\|_2 \implies \frac{\|\hat{v}-v\|_2}{\|v\|_2} \leq \|E\|_2$. Bound $\|E\|_2 \leq \|E\|_F = \sqrt{\sum_{i,j} E_{ij}^2} \leq \sqrt{4(2\epsilon_{mach} \max|Q_{ij}|)^2} = 4\epsilon_{mach}$

3. Connect the dots

Give an example of a dot product that is computed to low relative accuracy when using the standard algorithm in Julia.

The example aligns with the error analysis in the notes 09-10 on dot products: Formalizes this for dot products, computed via a loop $\mathbf{dot} = \sum_{k=1}^{n} x_k y_k$. It introduces a backward error model where the computed dot $\hat{v}_x$ satisfies $\hat{v}_x = v_x(1 + \sum_{j=m_{\min}}^{m_{\max}} \delta_j(1 + \eta_j) + O(\epsilon_{\text{mach}}^2))$, where $v_x = \sum_{k=1}^{n} x_k y_k$ is the exact dot, $|\delta_j|, |\eta_j| \leq \epsilon_{\text{mach}}$, and the error arises from both summation and multiplication roundoff. The computed result can be interpreted as $\hat{v}_x = \tilde{x}^T y$, where $\tilde{x}$ is a perturbed version of x with relative perturbations $O(\epsilon_{\text{mach}})$. However, if cancellation occurs, the forward relative error can be much larger than ϵ_{mach}, depending on the condition number $\kappa = \frac{|v_x|}{\sum|x_k y_k|}$.

Consider vectors $x = [10^8, 1.0, -10^8]$ and $y = [1.0, 1.0, 1.0]$, which we analyzed in a similar context. Exact dot product: $v_x = 10^8 \cdot 1 + 1 \cdot 1 + (-10^8) \cdot 1 = 1$. Sum of absolute products: $\sum |x_k y_k| = 10^8 + 1 + 10^8 = 2 \times 10^8 + 1$. Condition number: $\kappa = \frac{|v_x|}{\sum |x_k y_k|} \approx \frac{1}{2 \times 10^8 + 1} \approx 5 \times 10^{-9}$, indicating high sensitivity to perturbations.

```
let
    x = [1e8, 1.0, -1e8]
    y = [1.0, 1.0, 1.0]
    computed = dot(x, y)
    true_val = 1e8 + 1 - -1e8
    rel_err = abs(computed - true_val) / abs(true_val)
    println(computed, " ", true_val, " ", rel_err)
end
```

```
1.0 2.00000001e8 0.999999995
```

4. Back it up

Running the recurrence $E_n = 1 - nE_n$ forward is an unstable way to compute $\int_0^1 x^n e^{x-1}\,dx$. However, we can get good results by running the recurrence backward from the estimate $E_n \approx 1/(n+1)$ starting at large enough N.

- Implement this iteration starting from $E_{100} \approx 1/101$ to compute E_{20}. You should get good agreement with the code given (which computes the numbers in a completely different way).
- Let $\hat{E}_n$ be the floating point value computed by the backward recurrence and $f_n = \hat{E}_n - E_n$ be the error. Write a recurrence for f_{n-1}, which should take the form $f_{n-1} = (f_n + \gamma_n)/n$, where γ_n captures the roundoff.
- Using a uniform estimate of γ_n, augment your code to compute a running (estimated) error bound for the E_n computed by working backward from E_n. Your bound should incorporate both roundoff and the starting error. For controlling the starting error, you may use that $(n+1)^{-1} \le E_n \le (n+2)^{-1}$.
- How large should the starting point N be to compute E_{20} to near machine precision?

NB: `eps(Float64)` gives the distance between `1.0` and the next largest floating point number. This is twice the quantity that we refer to as ϵ_{mach} in class.

think it should be $E_n = 1 - nE_{n-1}$

1. Part 1. THe backward recurrence is $E_{n-1} = \frac{1 - E_n}{n}$ derived from $nE_n = 1 - E_{n-1}$. Then start at $N = 100$ with $E_{100} \approx 1/101$. See function En_backward()
2. Part 2. In floating point, the computed value is $\hat{E}_{n-1} = \text{fl}\left(\frac{1 - \hat{E}_n}{n}\right)$. Using the standard roundoff model, the floating-point operation introduces a relative error, approximated as $\hat{E}_{n-1} = \frac{1 - \hat{E}_n}{n}(1 + \delta)$, where $|\delta| \le \epsilon_{\text{mach}}$, and $\epsilon_{\text{mach}} \approx 2^{-52} \approx 2.22 \times 10^{-16}$ for double precision. The error is defined as $f_{n-1} = \hat{E}_{n-1} - E_{n-1}$. Substituting $f_{n-1} = \frac{1 - \hat{E}_n}{n}(1 + \delta) - \frac{1 - E_n}{n} = \frac{1}{n}\left[(1 - \hat{E}_n)(1 + \delta) - (1 - E_n)\right]$.

so, $f_{n-1} = \frac{1}{n}\Big[(1-\hat{E}_n+\delta-\delta\hat{E}_n)-(1-E_n)\Big]$. Then, $f_{n-1} = \frac{1}{n}\Big[E_n-\hat{E}_n+\delta-\delta\hat{E}_n\Big]$. Since $f_n = \hat{E}_n - E_n$, thus, $E_n - \hat{E}_n = -f_n$.

$$f_{n-1} = \frac{1}{n}\Big[-f_n+\delta(1-\hat{E}_n)\Big]$$

Let $\gamma_n = \delta(1-\hat{E}_n)$ represent the roundoff error term, where $|\gamma_n| \leq |\delta| \cdot |1-\hat{E}_n|$. Given that $\hat{E}_n$ is an approximation to E_n (which is positive and less than 1 for $n \geq 0$), and typically $|\hat{E}_n| \leq 1$, the term $1-\hat{E}_n$ is bounded.

Thus, the correct recurrence is $f_{n-1} = \frac{-f_n+\gamma_n}{n}$ where $\gamma_n = \delta(1-\hat{E}_n)$ captures the roundoff error.

3. Part 3. Assume a bound B_k backward alongside the recurrence. Starting from the initial approximation error at N: $B_N = \frac{1}{(N+1)(N+2)}$ (from the asymptotic expansion $E_N = \frac{1}{N+1} - \frac{1}{(N+1)(N+2)} + O(\frac{1}{N^3})$). At each step, since $f_{k-1} = \frac{-f_k+\gamma_k}{k}$, the absolute bound is $B_{k-1} \leq \frac{B_k+|\gamma_k|}{k}$. Using $|\gamma_k| \leq 2\epsilon_{mach}$ in double precision. See function backward_bound()
4. Part 4. The machine epsilon is in double precison, $\epsilon_{mach} \approx 2.220446049250313e-16$. The error bound increases as N decreases. Need to ensure $B_{20} < \epsilon_{mach}$ through the backward recurrence. See computation at the last block of code, N has to be start at 30.

```
En_ref (generic function with 1 method)
```

```
# Compute En by integrating a truncated Taylor series term by term
function En_ref(n)
    s = 0.0
    for j = 20:-1:1
        s = (s+1.0/(n+j+1))/j
    end
    s += 1.0/(n+1)
    s/exp(1.0)
end
```

```
En_backward (generic function with 1 method)
```

```
function En_backward(N, n)
    E = 1.0 / (N +1)
    for k  = N: -1:n+1
        E = (1.0-E) / k
    end
    E
end
```

```
let
    n = 20
    E20_ref = En_ref(n)
    E20_backward = En_backward(100, n)
    println(E20_ref, " ", E20_backward, " ", isapprox(E20_ref, E20_backward))
end
```

```
0.045544884075818054 0.045544884075818054 true
```

```
backward_bound (generic function with 1 method)
```

```
function backward_bound(N, n)
    E = 1.0 / (N + 1)
    B = 1.0 / ((N+1)*(N+2))
    gamma = 2 * eps(Float64)
    for k = N: -1:n+1
        E = (1.0-E) / k
        B = (B + gamma) / k
    end
    E, B
end
```

```
let
    E20, bound = backward_bound(100, 20)
    println("E20 = ", E20)
    println("Error bound = ", bound)
    println(bound < eps(Float64))
end
```

```
E20 = 0.045544884075818054
Error bound = 2.21519437838219e-17
true
```

```
let
    N = 30
    n = 20
    check_bound = backward_bound(N, n)[2]
    println(eps(Float64))
    println(check_bound < eps(Float64))
end
```

```
2.220446049250313e-16
true
```